WowFit Challenges — Week 2 Report

Planning, Setup & MVP Development

> ▱ = *to be filled by the team before submission (names / links / screenshots — things produced as work happens).*

---

1. Team & Roles

- **Team name:** Green Team
- **Communication channel:** Telegram — @brainonblueprint (team lead)
- **Repository:** https://github.com/Inno-Fitnesss/Fitness-Challenge-App

Member	Mail	Role	Responsibilities
Ivan	i.lokhmotov@innopolis.university	Team Lead, Backend	Planning, reviews, product decisions, coordination
Kristina	kr.ushakova@innopolis.university	Frontend	React app, UI, API integration
Elvina	e.belorusova@innopolis.university	CV (client), ML assistant	MediaPipe layer: camera, skeleton, rep counter
Daria	d.komzolova@innopolis.university	Backend	FastAPI, DB, challenge/streak/leaderboard logic
Safina	s.mukhamedshina@innopolis.university	ML	Recognition research (k-NN over pose embeddings, etc.), architecture
Ekaterina	e.salazkina@innopolis.university	ML/CV (assistant)	Data labeling, hypothesis testing, CV/ML support
Anastasia	a.kalashnikova@innopolis.university	Frontend/Backend, DevOps	Markup, backend, deployment
Timur	t.sagdeev@innopolis.university	Docs, Repository	Repo structure, descriptions, documentation

---

2. Project & Track Statement

Project

WowFit Challenges is a mobile-first web app where users join fitness challenges and confirm exercises through the camera. Pose recognition runs **in the browser** — the video never leaves the device; reps are counted in real time, the skeleton is drawn, and form is checked. Motivation is driven by a daily habit (the streak / "flame") and competition inside a challenge. In spirit it is "Duolingo for sport". MVP exercises: squats, push-ups, plank.

Track: **Industrial**

Problem Statement

WowFit is an existing platform with live training sessions and 180+ trainers. Its gap: **no engagement between sessions with a trainer** and no tools for self-guided workouts. A user trains only during a paid session; in between there is silence, which lowers retention and the value of the subscription. A cheap, high-reach product is needed to keep the habit of training **between** sessions.

Justification against Industrial acceptance criteria (point-by-point)

- **Existing product/project:** WowFit — a working live-training platform (180+ trainers). A backend partially exists (auth: signup/login/JWT).
- **Gap / issue / opportunity:** there is no self-guided engagement layer between sessions. No tools to train without a trainer, no gamification, no reason to open the product daily.
- **Non-trivial contribution:**
    1. **In-browser CV** on MediaPipe BlazePose — rep counting by joint angles + **explainable** form checking via geometry rules (highlight the wrong segment, text hints). Privacy by design: the video never leaves the device.
    2. **Retention gamification** — a daily streak ("flame"), scheduled challenges, a leaderboard by number of completed days; social layer (friends' flames) in V2.
    3. **Production-grade backend** — server-side computation of all progress (anti-cheat), versioning, migrations, security; realtime leaderboard in V2.
- **Potential impact beyond grading (qualitative):** gives WowFit a self-guided layer between sessions and habit gamification — potential for retention and subscription value, and a base to extend to new exercises. *We do NOT measure business metrics (retention/DAU)* — this is a summer course project; our measurable contribution is technical (see below).

Baseline & measurement (Industrial)

> *Summer course project — we measure what we actually control (quality of our contribution), not WowFit's business metrics.*

- **Baseline:** today a user has no self-training tool with **objective** checking — they count reps by eye, with no feedback on form.
- **What we measure (our contribution vs baseline):**
    1. **Rep-counting accuracy:** recall ≥ 85% against a manual ground-truth count (squats, push-ups).
    2. **Form-check quality:** share of correctly caught "dirty" reps on hand-labeled examples.
    3. **CV latency:** ≤ 100 ms on an average laptop in Chrome.
    4. **Product works:** an end-to-end challenge with 5+ participants on a public URL.
- **When:** baseline CV metrics on the rep-counting prototype (W3–W4), final check in W6.

---

3. Project Plan

Week-by-week roadmap (by role)

> *6-week project (~4 weeks MVP + 2 weeks V2). Week 1 ≈ the current course week.*

Week	Backend	Frontend	CV	ML
1	Setup, auth, DB/migrations	Setup, import into code, skeleton	Camera + skeleton drawing	Research kickoff
2	Challenge logic (1 challenge = 1 exercise):	Desktop + mobile,	Working rep counter	Research recognition without hard rules (k-NN

	create, `join_code`, schedule, participations, accept session	challenge and session screens		over pose embeddings?), data labeling, test approaches
3	Streak logic + leaderboard logic	Challenges, app + camera onboarding	Improve rep counter	Pick a model, build a service
4 — 🎯 MVP	Polish challenge logic; deploy; criteria	Polish challenge logic (UI); deploy	Stabilize 3 exercises	Finish service
5 (V2)	Polish, security	Polish	Form check (good/sloppy)	Per results
6 (V2)	Friends, public challenges	Friends, public challenges	—	—

Major milestones

- **M1 (W1):** Working auth + DB with migrations + camera/skeleton.
- **M2 (W2):** Working rep counter + challenge logic (create / join by link).
- **M3 (W4) — MVP core:** end-to-end flow (join → session → day closed → streak/leaderboard) + 3 exercises + ML service, on a public URL.
- **M4 (W6):** Friends + public challenges + criteria check (recall ≥85%, latency ≤100 ms, challenge with 5+).

Dependencies

- Form check depends on a stable rep counter (joint angles) → we do the counter first.
- Streak/leaderboard logic depends on the session and participation models.
- Deployment depends on the Docker infra (prepared in week 1).
- The ML service depends on labeled data (labeling in week 2).

Known risks

Risk	Impact	Mitigation
User doesn't understand how to place the camera (angle/distance)	Reps not counted → frustration	**Onboarding is our responsibility:** show technique the first time, live auto-check (camera+skeleton, auto-ticks "whole body in frame"), fixed angle per exercise (squat — front; push-up/plank — side)
Poor camera / lighting quality	Noisy skeleton points, wrong counts	Lighting auto-check in onboarding (only blocks on a problem), hints "add light / step back", normalization by body proportions
Rep-counting recall < 85%	Criterion failed	Calibrate angle thresholds, test on laptop/Chrome
Pose latency > 100 ms	Criterion failed	CV in a Web Worker, light PoseLandmarker model, throttle drawing
Not finishing the core by W3–W4 (tight time)	MVP slip	Hard split core / polish / V2 (see §4); friends and form check go to polish, not the core

Cheating progress via the console	Unfair leaderboard	All computation is server-side only; the client sends raw facts

Backlog (Kanban)

🔲 Board link. Prioritized MVP epics: **Auth** → **DB/migrations** → **CV rep counter** → **Challenges (by link) + presets** → **Session** → **Day-close/Streak** → **Leaderboard by days** → **Onboarding** → **Deploy**. In parallel — **ML research**. V2: **Friends** → **Public challenges** → **Form check** → **Realtime**.

---

4. Requirements

User Stories

Auth

- As a new user, I want to register with email/password so that I get an account.
- As a user, I want to log in and stay logged in so that I don't enter my password every time.

Challenges

- As a user, I want to join a challenge by link/code so that I can train.
- As a user, I want to create a challenge (exercise + schedule) and share it by link.
- As a beginner, I want to pick a ready-made workout preset (beginner/intermediate/advanced) so that I can start without setup.
- As a user, I want to see "Today's plan" on Home — the challenges scheduled for today.

Session / CV

- As a user, I want to perform an exercise under the camera and see the rep counter and skeleton so that I understand my progress.
- *(V2)* As a user, I want a hint about what's wrong with my form (segment highlight + text) so that I can fix it.

Progress

- As a user, I want to keep my daily flame by doing at least one scheduled workout per day so that I don't lose the streak.
- As a user, I want to see all-time rep counters in my profile.

Competition / social

- As a challenge participant, I want to see the ranking so that I can compete.
- *(V2)* As a user, I want to see friends' flames on Home so that we keep each other motivated.

Acceptance criteria (key)

Closing a challenge day (CV gate):

- In the MVP one challenge = one exercise; a day is closed **only** if the exercise reaches the daily goal with **clean** reps (passing the form rules).
- On close: `+1` to `days_completed` and to the challenge streak; clean reps go into the all-time counters.
- A partial / dirty day does not close the day.

Flame (global streak):

- +1 if **at least one** scheduled challenge day is closed that day.
- Resets to 0 if nothing is closed by local midnight.

- A day with no scheduled sessions is neutral (neither grows nor breaks the streak).

Challenge leaderboard:

- Ranked by `days_completed` within that challenge.
- No tie-breakers — ties are allowed (several people can share 1st place).
- Updates on entry / after a session (realtime via WS — in V2).

Server-side computation (anti-cheat):

- The client sends only raw facts (`total` , `clean` , `duration`); the server computes streak / rank / day-close.

MVP Scope

MVP core (~4 weeks, no friends):

- Auth (working sign up / sign in), `username` for display.
- Rep counting: start with 1 exercise (squat) → grow to 3.
- Challenges: **closed** (by link/code), **one challenge = one exercise**, schedule (basic activity), join + create + share.
- **Ready-made challenge presets** from the app (beginner/intermediate/advanced).
- Quality onboarding (show technique + camera calibration).
- Today's plan on Home, camera session, day close (quality gate), flame streak (cartoon style).
- Challenge leaderboard — "who has trained how many days, what streak".
- Profile: own flame + all-time counters + archive (frozen challenges) + edit.
- Deploy to a public URL (the university VM is already provided).
- End of MVP: onboarding polish (light/frame auto-check) + auth security fixes.

V2 — in-course polish (weeks 5–6):

- **Friends:** friendship, challenge invites, flames on Home, highlighting friends in the leaderboard.
- **Public challenges.**
- Deeper form check (good/sloppy, segment highlight, text hints).
- Realtime leaderboard (WS + Redis), frozen podium of a finished challenge.

OUT (Future, out of course scope):

- Search/catalog of public challenges, join requests (friend-gated), captcha.
- Weekly individual league (XP), points/multiplier system, workout deferral.
- Multiple exercises in one challenge.

---

5. Tech Stack (with justification)

Layer	Choice	Why
Frontend	React + TypeScript, Vite, mobile-first (Tailwind)	One adaptive web app; TS for contract type-safety
State/routing	React Context + react-hook-form + React Router	Lightweight state; CV layer decoupled from UI via events
CV	MediaPipe Tasks Vision (PoseLandmarker / BlazePose) in a Web Worker	The heavy job (pixels→33 points) is solved; a worker keeps latency ≤100 ms; privacy (video stays in browser)

Backend	Python **FastAPI** + Pydantic	Existing auth is already on the FastAPI stack; fast, async, native WebSockets
ORM/migrations	SQLAlchemy (+ Alembic planned)	Production requirement: DB migrations
Auth	JWT (PyJWT) + bcrypt	Production tokens; security fixes tracked internally
DB	PostgreSQL	Source of truth (users, challenges, sessions, streaks)
Realtime / cache *(V2)*	Redis + WebSockets	Live leaderboard state + revoked tokens. In the MVP — simple refresh
ML (active track)	Research (k-NN over pose embeddings, etc.)	A separate ML team: recognition/form quality, data labeling, hypothesis testing
Infra	Docker + docker-compose, Nginx, public URL	Success criterion is a deployed public URL; one command runs all services

---

6. Project Setup

Repo structure (current)

```
Fitness-Challenge-App/
├── frontend/            # React + TS + Vite (auth UI), Nginx Dockerfile
├── backend/             # FastAPI (auth: signup/login/JWT), Dockerfile
├── docs/                # this report, use-case diagram, API PDFs
├── docker-compose.yml   # frontend + backend + db (postgres)
├── .gitignore
└── README.md
```

Subfolders for challenges/sessions/CV/ML are added in the following weeks.

docker-compose (services)

`frontend` (Nginx static + `/api` proxy) · `backend` (FastAPI/uvicorn) · `db` (postgres). One `docker compose up --build` starts everything; secrets via environment. Redis — in V2.

Runnable boilerplate

- Backend: FastAPI with `GET /health` and the auth router.
- Frontend: React app served by Nginx, proxying `/api` to the backend.
- README with end-to-end run commands.

---

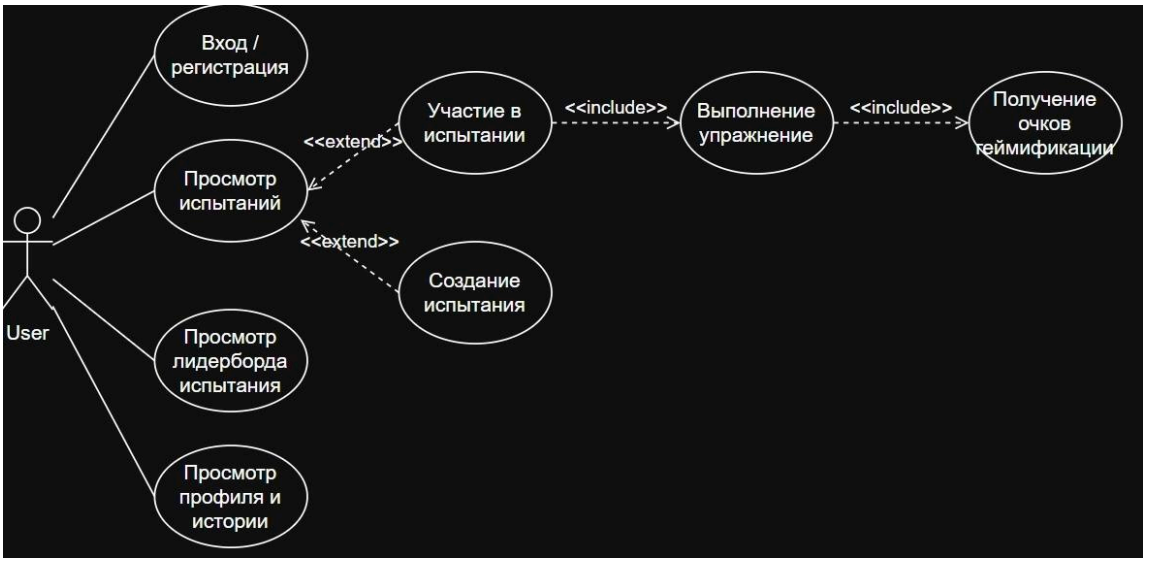
7. Initial Design & Skeleton

Design (Figma)

https://www.figma.com/design/K9D2ZPXKtQ3zCleTb3TXtH/SAAS-Dashboard--Community-?node-id=2608-2 (may change)

Existing mockups: Registration, Login, Home, Challenge screen, Challenge list. Edits and missing screens (Session, Result, Profile, Friends, Password reset, Create challenge) are tracked in the internal product spec. User flow: Registration → Onboarding → Home (plan) → Session (camera) → Result → Challenge leaderboard.

Use-case diagram (UML)



Actors and scenarios: **User** → Login/registration · View challenges (→ extend: Join challenge, Create challenge) · Join → include: Perform exercise → include: receive progress reward (flame/rank — *not XP points*) · View challenge leaderboard · View profile and history.

Frontend skeleton

Tab bar (3 tabs: Home · Challenges · Profile), routing, skeleton screen components. Auth screens (Sign in / Sign up) implemented.

Backend skeleton + API contract

- Existing auth (signup/login/JWT) on FastAPI; security and Users-schema fixes are tracked internally.
- **API docs (interim):** `docs/api/API Documentation.pdf` + `docs/api/Database Structure.pdf` (manual). **Next:** enable the auto-generated **Swagger/OpenAPI** from FastAPI (`/docs` , `/openapi.json`) — it always matches the code and shows progress (number of endpoints).
- Entity outline (exact schema designed separately): `users` (+ `username`), `exercises` , `challenges` (closed, `schedule`), `participations` , `sessions` . Friendships/public — V2.
- OpenAPI draft `/v1` : `/auth/*` , `/me/today` , `/challenges` , `/challenges/{id}/join` , `/sessions` , `/challenges/{id}/leaderboard` , `/me/profile` .

ML / experimentation setup (track-appropriate)

- **Approach:** BlazePose (MediaPipe) gives 33 points; on top — research of recognition/quality scoring (angle algorithm vs **k-NN over pose embeddings**, etc.).
- **Dataset plan:** collect and **label** short clips of 3 exercises (squat/push-up/plank), good/sloppy reps; manual ground-truth count for recall.
- **Experimentation env:** notebooks (`.ipynb`) in `ml/` or a separate repo section; compare approaches against the algorithmic baseline.
- **Baseline/metrics:** rep recall ≥ 85%, latency ≤ 100 ms, share of caught "dirty" reps.

- Links to notebooks/dataset/commits — as they appear.

---

8. Core MVP Implementation (status this week)

Done: working authentication end-to-end. Frontend is connected to the backend; the whole stack starts with a single `docker compose up --build`.

- **Implemented features:** registration (`POST /auth/signup`), login with JWT (`POST /auth/login`), protected route (`GET /protected`), health check (`GET /health`). The user is stored in PostgreSQL. ☐ UI screenshots/GIF.
- **Functional user journey:** open the frontend → register → log in → receive a token (stored in localStorage, sent in `Authorization`). Verified manually: signup creates a user (`id:1`), login returns a JWT.
- **Frontend↔API↔DB link:** ✅ works. Frontend (React/Vite behind Nginx) → `/api` → Backend (FastAPI) → PostgreSQL. Confirmed: `GET localhost:3000/api/health` is proxied to the backend and returns `{"status":"Running...."}`.
- **Infra:** `docker compose` starts 3 services (frontend, backend, db); the backend reads DB and secrets from env, CORS is configured, tables auto-create on startup.
- **Cleanup:** the research branch `feature/cv-test` (Elvina) was removed from `main` and kept as a separate branch.

---

9. Individual Contributions

☐ To be filled per person before submission. For each — concrete deliverables by role type:

Member (role)	Done this week	Deliverables (links)
Ivan — Team Lead, Backend	☐	Issues/Kanban ☐ · PRs ☐ · TA feedback + action items ☐
Kristina — Frontend	☐	Merged PRs/issues ☐ · UI screenshots/GIF ☐
Elvina — CV, ML assistant	☐	PRs/commits ☐ · rep-counter GIF ☐
Daria — Backend	☐	Merged PRs/issues ☐ · Swagger link ☐
Safina — ML	☐	Commits/`.ipynb` ☐ · dataset/model artifacts ☐
Ekaterina — ML/CV assistant	☐	Labeling/`.ipynb` ☐ · hypothesis results ☐
Anastasia — Frontend/Backend, DevOps	☐	PRs ☐ · deploy/Docker on VM ☐
Timur — Docs, Repository	☐	README/docs ☐ · repo structure (PR) ☐

---

10. Links

- Repository (master — working code): https://github.com/Inno-Fitnesss/Fitness-Challenge-App
- Deployed on VM (URL for TAs): ☐
- PRs/Issues (skeleton + features): ☐
- API docs: interim — `docs/api/API Documentation.pdf` ; Swagger from FastAPI — ☐
- Kanban board: ☐

- Figma: see §7
- ML notebooks/dataset: ☐
- Video demo of the week: ☐

---

11. Internal Demo & Next Steps

☐ Demo of the current state:

- What works: ☐
- Bugs found: ☐
- Immediate improvements: ☐

Plan for Next Week (Week 3)

- **Backend:** streak + leaderboard logic; product `/v1` endpoints.
- **Frontend:** Challenges screen, app + camera onboarding. Deliverable: merged PRs + screenshots.
- **CV:** improve the rep counter (recall stability).
- **ML:** pick a model/approach, start the recognition service. Deliverable: `.ipynb` + first results.
- **General:** keep master working + deployed on the VM; update the Kanban; record a video demo.